

Elastic Scheduler

Overview

The Elastic Scheduler runs on top of Apache Spark in the Amazon Elastic Map Reduce (EMR) cluster platform. Input data is read from and the results are written to an S3 bucket. We used EMR version emr-6.13.0, and each of the primary, core and task nodes is of type m5.xlarge with 4 virtual cores and 16 GiB memory.

Datasets

- TPC-H
 - Dataset generation is done using the tpch-dbgen tool (Details in https://support.hpe.com/hpesc/public/docDisplay?docId=sf000078704en_us&docLocale=en_US)
 - Split the generated data into 4500 files of Orders and Lineitem using Dataset/TPC-H/TpchFileGenerator.java
 - Dataset modified to add timestamp using Dataset/TPC-H/TpchTimestampUpdate.java
- Yahoo Streaming
 - Dataset/YahooStreaming/DatasetGen.java

Code Organisation

Brief description of each of the classes is given below

1. App.java
 - Contains the main class which invokes the appropriate modules in QueryScheduler
2. QueryScheduler.java
 - This is the Query Scheduler component which carries out the main functionalities such as
 - a. Run Simulation
 - i. Checking for addition/deletion in queries
 - ii. Computing the actual input rate
 - iii. Invoking simulation when needed

- b. Schedule Execution
 - i. Determines the readiness of query batches based on the input availability
 - ii. Sorts the queries which are ready
 - iii. Schedules the query with least slack for processing
 - iv. Issues node resize requests based on the generated schedule
3. ScheduleOptimiser.java

This class implements the Schedule Optimiser component which does the following

 - a. Simulates Query scheduling with elastic configuration and generates query batch schedule containing the list of query batches scheduled of execution at different time points along with required number of nodes
 - b. Simulates Query scheduling with other modes such as fixed configuration, LLF without batch size determination for comparison with our approach.
4. Query.java

This class represents a single query with its attributes such as query ID, window start and end time, deadline, input rate, cost model etc

It contains functions for computing the duration for processing a given set of tuples, aggregation time computation and slack time computation based on the query's cost model.
5. QueryRepository.java

This class contains the Spark processing functions for each of the TPC-H, Custom queries and Yahoo Streaming query, including the partial and final aggregation processing modules.
6. Cluster.java

This class contains the attributes and functions specific to the cluster platform, (i.e. AWS EMR related as per our implementation) such as functions to issue node resize requests, get the current instance count etc.
7. CSSparkSession.java

This class creates the Spark session and the context
8. Logger.java

This class contains modules that write the execution log / results to a flat file.

Input /Output Files

- Input dataset is accessed from AWS S3 buckets with the following directory structure
- TPC-H Input data
 - s3 ---> <s3 Bucket Name> ---> input ---> file-ip ---> static --->
 - customer
 - nation
 - parts
 - partSupp
 - region
 - supplier

- s3 ---> <s3 Bucket Name> ---> input ---> file-ip ---> streaming --->
 - orders
 - lineitem
- Yahoo Streaming Input data
 - s3 ---> <s3 Bucket Name> ---> yahoodataset
 - staticdata
 - jsonevents
- Query Output
 - s3 ---> <s3 Bucket Name> ---> output ---> results
 - tmp2 (contains the intermediate results)
 - Directory for each query
 - tmp3 (contains the final results)
 - Directory for each query
 - tmp4 (contains the partial aggregation results)
 - Directory for each query
- Query attributes
 - CustSch/query-attr.txt
- Input rates
 - CustSch/ipRate.txt
- Case wise Query attributes and Ip rates files

SI No.	Dataset	Case	Query Attributes	Input Rate
1.	TPC-H	1D	query-attr-1d.txt	fixedrate.txt
2.		0.8D	query-attr-0.8d.txt	fixedrate.txt
3.		0.6D	query-attr-0.6d.txt	fixedrate.txt
4.		0.4D	query-attr-0.4d.txt	fixedrate.txt
5.		0.3D	query-attr-0.3d.txt	fixedrate.txt
6.		2FR	query-attr-1d-2.txt	fixedrate-2.txt
7.		4FR	query-attr-1d-4.txt	fixedrate-4.txt
8.		VR1	query-attr-1d-2.txt	awsVarRate1.txt
9.		VR2	query-attr-1d-2.txt	awsVarRate2.txt
		LLF without Batch size determination	query-attr-1d.txt	fixedrate.txt
10.	Yahoo Streaming	1FR:1D	yquery-attr-1d.txt	yfixedrate.txt
11.		1FR:0.4D	yquery-attr-0.4d.txt	yfixedrate.txt

12.		1FR:0.2D	yquery-attr-0.2d.txt	yfixedrate.txt
13.		2FR:1D	yquery-attr-1d-2.txt	yfixedrate-2.txt
14.		4FR:1D	yquery-attr-1d-4.txt	yfixedrate-4.txt
15.		6FR:1D	yquery-attr-1d-6.txt	yfixedrate-6.txt

- Environment Variables (ENVs) EVNs
 - TPC_INPUT_PATH = s3://<s3 Bucket Name>/input/
 - YAHOO_INPUT_PATH = s3://<s3 Bucket Name>/yahoodataset/
 - OUTPUT_PATH = s3://<s3 Bucket Name>/output/
 - QRY_INPUT_PATH = <Absolute path to CustSch directory>

Execution

Elastic Scheduler can be invoked in one of the following modes

1. Simulation LLF
 - Copy query-attr-1d.txt to query-attr.txt in \$QRY_INPUT_PATH
 - Copy fixedRate.txt to ipRate.txt in \$QRY_INPUT_PATH
 - Run spark-submit --class "SchIQP.CustSch.App" CustSch/target/CustSch-0.0.1-SNAPSHOT.jar SimLLF
 - Results are available in \$QRY_INPUT_PATH/exelog.txt
2. Simulation Fixed Configuration
 - Copy required query attribute file to query-attr.txt in \$QRY_INPUT_PATH
 - Copy the required input rate to ipRate.txt in \$QRY_INPUT_PATH
 - Run spark-submit --class "SchIQP.CustSch.App" CustSch/target/CustSch-0.0.1-SNAPSHOT.jar SimFC
 - Results are available in \$QRY_INPUT_PATH/exelog.txt
3. Simulation Elastic Configuration
 - Copy required query attribute file to query-attr.txt in \$QRY_INPUT_PATH
 - Copy the required input rate to ipRate.txt in \$QRY_INPUT_PATH
 - Run spark-submit --class "SchIQP.CustSch.App" CustSch/target/CustSch-0.0.1-SNAPSHOT.jar SimEC
 - Results are available in \$QRY_INPUT_PATH/exelog.txt
4. Experimental mode
 - Copy required query attribute file to query-attr.txt in \$QRY_INPUT_PATH
 - Copy the required input rate to ipRate.txt in \$QRY_INPUT_PATH
 - Update EMR cluster ID in \$QRY_INPUT_PATH/[cluster.sh](#). (\$QRY_INPUT_PATH/[cluster.sh](#) must have execute permissions)

- Run spark-submit --class "SchIQP.CustSch.App"
CustSch/target/CustSch-0.0.1-SNAPSHOT.jar Exp <EMR TASK Instance group
ID>
- Results are available in \$QRY_INPUT_PATH/res.txt